# Project Proposal

## EdgeCase Foundry — AI-Powered Edge Case Generator and Software Testing Assistant

**Submitted By:** [Your Name]

**Degree Program:** BS Data Science

**University:** Superior University, Lahore

**Project Type:** Generative AI Web Application

### 1. Introduction

Software applications can contain hidden errors that appear only under unusual conditions. For example, an application may fail when the internet connection is interrupted, the same information is submitted multiple times, or a user enters unexpected data. These situations are often called *edge cases*.

Finding such cases manually can be time-consuming, especially for developers who need to test many different situations. EdgeCase Foundry is a Generative AI-powered web application designed to help solve this problem by automatically generating unusual test scenarios from a simple description of a software system.

### 2. Problem Statement

Traditional software testing often focuses on normal user activities and common inputs. As a result, unusual situations and unexpected system behaviors may remain untested until they cause real problems.

Developers may spend considerable time identifying edge cases, preparing test steps, and deciding what the expected results should be. There is a need for an intelligent tool that can assist developers in discovering potential weaknesses and preparing structured test cases more efficiently.

### 3. Proposed Solution

EdgeCase Foundry will allow users to describe their software system in simple English. Using a Generative AI model, the application will analyze the description and generate relevant edge cases and test scenarios.

For each scenario, the system will provide a description of the situation, its category, risk level, testing steps, expected result, and a suggested solution or improvement.

The generated results will be organized in an interactive dashboard, allowing users to review, filter, prioritize, and export test cases. The application will assist with test planning; actual verification of software behavior will require a runnable application or a suitable testing environment.

## 4. Objectives

The main objectives of this project are:

- To develop a web application that generates edge cases using Generative AI.
- To identify potential problems involving invalid inputs, boundary values, network failures, duplicate submissions, and unexpected user actions.
- To generate clear and structured software test cases.
- To classify scenarios according to their risk and importance.
- To provide expected results and possible solutions for each test case.
- To allow users to export generated test cases and reports.
- To demonstrate how Generative AI can improve software testing and quality assurance.

## 5. Key Features

### 5.1 System Description Analyzer

The user describes a software system, and the application identifies its main functions, assumptions, and possible failure points.

### 5.2 AI Edge Case Generator

The AI generates multiple unusual test scenarios based on the system description, including invalid inputs, boundary conditions, network problems, duplicate entries, and simultaneous user actions.

### 5.3 Risk Prioritization

Each scenario receives a risk level, such as Low, Medium, High, or Critical, using defined criteria to help users focus on important cases first.

### 5.4 Test Case Builder

For each edge case, the system generates the scenario description, preconditions, test steps, input conditions, expected result, and suggested improvement.

### 5.5 Interactive Dashboard

Users can view generated test cases, filter them by category or risk level, and explore summaries and charts.

### 5.6 Report Export

Users can download their generated test cases as CSV files and export a formatted PDF report.

## 6. Example Scenario

Suppose a user describes a university attendance system that records attendance using student ID cards.

EdgeCase Foundry might generate the following test cases:

| Test Scenario | Expected Result |
| --- | --- |
| A student scans the same card twice within a few seconds. | The system should prevent duplicate attendance records. |
| The internet disconnects after a card is scanned. | The system should handle the failure without losing or incorrectly duplicating the record. |
| An invalid student ID | The system should |

| | |
|---|---|
| is scanned. | reject the ID and display an appropriate message. |
| Two devices scan the same student's card simultaneously. | The system should record attendance consistently without creating duplicate entries. |
| A student scans their card at the attendance deadline. | The system should apply the defined attendance policy correctly. |

These examples illustrate the type of test cases the application will generate. Their correctness and relevance will be reviewed before use.

## 7. Tools and Technologies

The project will use the following technologies:

- **Programming Language:** Python

- **Web Application Framework:** Streamlit
- **Generative AI:** A suitable free-tier LLM API or a locally running open-source model
- **Data Validation:** Pydantic
- **Data Processing:** Pandas
- **Visualization:** Plotly
- **Database:** SQLite, if project history and saved reports are included
- **PDF Report Generation:** ReportLab
- **Version Control:** Git and GitHub

The final AI provider will be selected based on free-tier availability, usage limits, response quality, and data privacy requirements.

## 8. Methodology

The project will be developed in the following stages:

1. **Requirement Analysis:** Define the target users, system inputs, output format, and supported categories of edge cases.
2. **Interface Development:** Build a simple web interface where users can enter a software description and select testing preferences.
3. **AI Integration:** Connect a Generative AI model to analyze descriptions and generate structured test scenarios.
4. **Output Validation:** Validate the generated results, remove obvious duplicates, and ensure required fields are present.
5. **Risk Classification:** Apply a consistent scoring rubric to prioritize generated scenarios.
6. **Dashboard Development:** Display test cases, risk summaries, and category-based charts.
7. **Report Generation:** Implement CSV export and PDF report generation.
8. **Testing and Evaluation:** Test the application with different example systems and assess the relevance, clarity, and usefulness of the generated cases.

## 9. Project Scope

The initial version will focus on generating test cases for software systems described in plain English. It will support multiple categories of edge cases, risk prioritization, an interactive dashboard, and report export.

The first version will not automatically execute tests against every type of real software application. Automated execution may be considered as a future enhancement if time and resources allow.

## 10. Expected Outcomes

At the end of the project, a working web application is expected to be developed that can:

- Accept a description of a software system.
- Generate approximately 10–20 relevant edge cases per request, depending on model output and configured limits.
- Organize test scenarios into understandable categories.
- Provide risk levels, test steps, expected results, and suggested improvements.
- Display useful summaries and charts.
- Export test cases and reports for further use.

The project will also demonstrate the practical application of Generative AI, structured data validation, Python programming, and data visualization in software quality assurance.

## 11. Project Timeline

| Week | Planned Work |
| --- | --- |
| Week 1 | Requirement analysis, project design, and interface development |
| Week 2 | Generative AI |

| | |
|---|---|
| | integration and edge case generation |
| Week 3 | Output validation, risk scoring, and dashboard development |
| Week 4 | CSV/PDF exports, testing, evaluation, and final documentation |

## 12. Conclusion

EdgeCase Foundry aims to make software testing more efficient by using Generative AI to discover unusual situations that developers might overlook during manual testing. By converting simple software descriptions into structured test scenarios, the application will help users identify potential risks, plan better tests, and improve software reliability. The project is suitable for a Data Science student because it combines Generative AI, Python development, structured data processing, risk analysis, and visualization in a practical real-world application.